DBMS LAB RECORD SUBMISSION

CONTENTS

Experiment No	Title	Page No
1	DESIGN A DATABASE SCHEMA FOR AN APPLICATION	
2	FAMILIARISATION OF DDL COMMANDS	
3	DATABASE INITIALIZATION AND DATA IMPORT TO A DATABASE	
4	FAMILIARISATION OF DML COMMANDS	
5	IMPLEMENTATION OF AGGREGATE FUNCTIONS, ORDER BY, GROUP BY & HAVING CLAUSE IN SQL	
6	IMPLEMENTATION OF SET OPERATORS NESTED QUERIES, AND JOIN QUERIES	
7	IMPLEMENTATION OF TCL AND DCL COMMANDS	
8	VIEWS	
9	PROCEDURES, TRIGGERS AND FUNCTIONS	
10	PACKAGES AND CURSORS	
11	CRUD OPERATIONS ON A CASSANDRA TABLE	
12	CQL QUERIES ON CASSANDRA TABLES FOR DATA RETREIVAL	
13	APPLICATION DEVELOPMENT USING MONGODB WITH PYTHON	

Date : **Page No:**

Experiment No.1

DESIGN A DATABASE SCHEMA FOR AN APPLICATION

AIM

Design a database schema for an application with ER diagram from a problem description given below.

- A company maintains a relational database to manage information about its employees, departments, and their dependents.
- The **EMPLOYEE** table stores details such as employee ID, name, date of birth, city, date of joining, salary (defaulting to 0 if not specified), and the department they belong to.
- The **DEPARTMENT** table keeps track of department ID, department name, location, and allocated funds.
- Each employee is associated with a specific department through a foreign key relationship.
- Additionally, the **DEPENDENT** table records information about individuals who depend on employees, including their name, age, gender (restricted to ‘Male’, ‘Female’, or ‘Others’), relationship with the employee, and the corresponding employee ID.

- Proper primary key and foreign key constraints ensure data integrity and maintain relationships among the tables, enabling efficient management of organizational and personal employee data.

ER Diagram

<paste ER diagram>

SCHEMA Diagram
<Paste schema diagram >

RESULT
Designed a database schema. ER diagram and Schema diagram drawn and verified.

Date : **Page No:**

Experiment No.2

FAMILIARISATION OF DDL COMMANDS

AIM
Creation of database schema - DDL (create tables, set constraints, enforce relationships, create indices, alter, delete and modify tables). Export ER diagram from the database and verify relationships (with the ER diagram designed in step 1).

QUERIES AND OUTPUTS

a. Create the three tables in the database schema designed in experiment 1.
<Paste screenshot of query and output>
b. Add a DEFAULT constraint on Fund in DEPARTMENT. The default value of Fund should be 1000/-
<Paste screenshot of query and output>
c. Change the datatype of Age in DEPENDENT from int to float.
<Paste screenshot of query and output>
d. Add a PhoneNumber column to EMPLOYEE table.
<Paste screenshot of query and output>
e. Drop the DEFAULT constraint on Fund in DEPARTMENT
<Paste screenshot of query and output>
f. Drop the PhoneNumber column in EMPLOYEE table.
< Paste queries and outputs of each question.

Then, Paste exported diagram from mysql workbench >

RESULT
Queries involving basic DDL commands were executed and outputs were verified

---

Date : **Page No:**

Experiment No.3

DATABASE INITIALIZATION AND DATA IMPORT TO A DATABASE

AIM
Perform Database initialization - Data insert, Data import to a database (bulk import using UI and SQL Commands).

QUERIES AND OUTPUTS

a. Database initialization and execute various database operations.

<Paste mysql workbench screenshots of create, insert, delete, select.

b. Bulk import using UI tool .

< Paste mysql workbench screenshots of'bulk import using UI'>

c. Bulk import using SQL commands.

<Paste screenshots of SQL bulkimport query and outputs of bulk import using SQL commands'>

RESULT

Database initialization and Data import to a database using UI and SQL Commands were executed and outputs were verified.

Date : **Page No:**

Experiment No.4

FAMILIARISATION OF DML COMMANDS

AIM

Practice SQL commands for DML (insertion, updating, deletion of data, and viewing/ querying records based on condition in databases).

QUERIES AND OUTPUTS

a. Insert atleast 5 records in each tables.

<Paste screenshot of query and output>

b. Change city of employee Anita to Bangalore.

<Paste screenshot of query and output>

c. Give a 30% increment in salary of all employees in department ID 2.

<Paste screenshot of query and output>

d. Delete a dependent named 'Raj'.

<Paste screenshot of query and output>

e. Display employees whose name starts with 'A'.

<Paste screenshot of query and output>

f. Display the name of all dependents whose age is greater than 60.

<Paste screenshot of query and output>

g. Find the name of employees who resides in either Kannur, Calicut or Trivandrum.

<Paste screenshot of query and output>

RESULT

Queries involving basic DML commands were executed and outputs were verified

---

Date : **Page No:**

Experiment No.5

IMPLEMENTATION OF AGGREGATE FUNCTIONS, ORDER BY, GROUP BY & HAVING CLAUSE IN SQL

AIM

Implement various aggregate functions, Order By, Group By & Having clause in SQL.

QUERIES AND OUTPUTS

a. Aggregate functions

1.Find total number of employees in the comapany.

<Paste screenshot of query and output>
2.Find maximum salary in the company.
<Paste screenshot of query and output>
3.Find youngest dependent age
<Paste screenshot of query and output>
4.Find total salary paid to all employees who joined the company in the year 2022.
<Paste screenshot of query and output>
5.Find average age of dependents of employee with Employee ID 5.
<Paste screenshot of query and output>

b.Order By, Group By & Having clause in SQL.

1. Find average salary of employees in each department.
<Paste screenshot of query and output>
2. Count the number of dependents for each employee.
<Paste screenshot of query and output>
3. List employees ordered by salary (highest first).
<Paste screenshot of query and output>
4. List employees ordered by Date of Joining (latest first).
<Paste screenshot of query and output>
5. Find departments having more than 5 employees.
<Paste screenshot of query and output>
6. Find departments where average salary is greater than 50,000.
<Paste screenshot of query and output>
7. List departments with more than 3 employees, ordered by average salary.
<Paste screenshot of query and output>

RESULT
Implemented various aggregate functions, Order By, Group By and Having clause in SQL and outputs were verified.

Date : **Page No:**

Experiment No.6

IMPLEMENTATION OF SET OPERATORS NESTED QUERIES, AND JOIN QUERIES

AIM
Implement set operators, nested queries, and join queries in SQL.
QUERIES AND OUTPUTS

a. SQL Set Operators
1. Combines employee names and dependent names into a single list.
<Paste screenshot of query and output>
2. List all cities where either employees stay or departments are located including duplicates.
<Paste screenshot of query and output>
3. Find cities where both employees live and departments are located
<Paste screenshot of query and output>
4. Display the names that appear both as employee names and dependent names
<Paste screenshot of query and output>
5. Display the Ids of the Employees who have no dependents
<Paste screenshot of query and output>

b. SQL Join operators

1. List employee name and department name of each employee.
<Paste screenshot of query and output>
2. Display the name of employees working in the ‘HR’ Department
<Paste screenshot of query and output>
3. List the employees whose dependents are older than 18
<Paste screenshot of query and output>
4. List the name of employees with more than one dependent
<Paste screenshot of query and output>
5. List all employees and dependents using left join
<Paste screenshot of query and output>
6. List all employees and their department using right join
<Paste screenshot of query and output>
7. Display all employees and all departments, including employees without departments and departments without employees.
<Paste screenshot of query and output>
8. List the name of employees and department names of those employees that have no dependents
<Paste screenshot of query and output>

c. SQL nested queries

1. List the name of employees who earn more than the average salary of all employees
<Paste screenshot of query and output>
2. List the name of employees working in the department named 'HR'
<Paste screenshot of query and output>
3. Display the name and salary of the employees who have at least one dependent.
<Paste screenshot of query and output>
4. Display the name, ID and DOJ of employees who do NOT have any dependents.
<Paste screenshot of query and output>
5. Display the name,DeptID and DOB of employee(s) with the highest salary in the company.
<Paste screenshot of query and output>
6. List the Employee names who earn more than every employee in departments located in ‘Delhi’
<Paste screenshot of query and output>
7. List the employee names who do NOT work in departments with fund less than company average fund.
<Paste screenshot of query and output>

RESULT

Implemented set operators nested queries, and join queries in SQL and outputs were verified.

Date : **Page No:**

Experiment No.7

IMPLEMENTATION OF TCL AND DCL COMMANDS

AIM

Practice of SQL TCL commands like Rollback, Commit, Savepoint, Practice of SQL DCL commands for granting and revoking user privileges.

QUERIES AND OUTPUTS

a. TCL commands

i. Start a new transaction. Change the salary of the employee whose EmpID is 5 as 50000/-. Undo the update operation. Commit the transaction and display the changes to the table.

<Paste screenshot of query and output>
ii. Start a new transaction. Delete a record from EMPLOYEE table. Then, insert a new record in DEPENDENT table. Undo the insert operation only at first. Then, undo the delete operation also. Commit the transaction and display the changes to the table.
<Paste screenshot of query and output>

b. ***DCL commands***
i. Grant insert, update privilege to the user. Then perform the operations to verify the privileges.
<Paste screenshot of query and output>
ii. Revoke update privilege from the user. Verify the same.
<Paste screenshot of query and output>

RESULT
Implemented queries that involves SQL TCL commands and DCL commands and outputs were verified.

Date : **Page No:**

Experiment No.8

VIEWS

AIM
Practice SQL commands for creation of views.

QUERIES AND OUTPUTS
SQL Views
i. Create a view that contains details of Employees staying in Bangalore.
<Paste screenshot of query and output>
ii. Create a view that contains Employee basic details with their department names
<Paste screenshot of query and output>
iii.Create a view that contains names of employees in CS department
<Paste screenshot of query and output>
iv. Drop the created views
<Paste screenshot of query and output>

RESULT
Implemented queries to create views in SQL and outputs were verified.

Date : **Page No:**

Experiment No.9

PROCEDURES, TRIGGERS AND FUNCTIONS

AIM
Create Procedures, Triggers and Functions in SQL.

QUERIES AND OUTPUTS
a. Procedures
1. Create a Procedure to Add New Employee.
<Paste screenshot of query and output>
2. Create a Procedure to Increase Salary of a particular Employee by a particular input amount.
<Paste screenshot of query and output>
3. Create a Procedure to Count Employees in a particular Department.

<Paste screenshot of query and output>
4. Create a Procedure to count the number of Employees with Salary Above Given Amount
<Paste screenshot of query and output>
5. Create a Procedure to Delete Employee Only If No Dependents
<Paste screenshot of query and output>
6. Create a procedure to transfer an Employee to Another Department

b. Triggers

1. Create a trigger that prevents the entry of an employee whose salary is less than 10,000
<Paste screenshot of query and output>
2. Create a trigger that limits maximum number of dependents per employee as 3.
<Paste screenshot of query and output>
3. Create a trigger that reduces department fund by the salary of the new employee when he/she is added.
<Paste screenshot of query and output>
4. Create a trigger that prevents salary reduction
<Paste screenshot of query and output>
5. Whenever an employee's salary is updated, store the old and new salary details in a log table. Create a trigger.
<Paste screenshot of query and output>
6. Before deleting an employee record from employee table, store it in a backup table.
<Paste screenshot of query and output>

c. Functions.

1. Create a function to get number of employees in a particular Department
<Paste screenshot of query and output>
2. Create a function to Calculate Average Salary of a Department
<Paste screenshot of query and output>
3. Create a function to Calculate Employee Age from DOB
<Paste screenshot of query and output>
4. Create a Function to get Details of an Employee with his/her Department details
<Paste screenshot of query and output>

RESULT

Implemented queries to create Procedures, Triggers and Functions in SQL and outputs were verified.

Date : **Page No:**

Experiment No.10

PACKAGES AND CURSORS

AIM

Create Packages and cursors in SQL.

QUERIES AND OUTPUTS

a. Packages

Create a schema named `company_pkg` to simulate a package.
<Paste screenshot of query and output>
i. Inside this schema, create a procedure to add a new employee.
<Paste screenshot of query and output>
ii. Inside this schema, create a procedure named `safe_delete_department` that accepts `DeptID` as input.

- Checks whether any employees exist in that department. If employees exist, raise an exception with an appropriate error message.
- If no employees exist, delete the department. Display a success message after deletion.

<Paste screenshot of query and output>

iii. Create a procedure that:

- Transfers employee to another department
- Reduces old department fund by 5%
- Increases new department fund by 5%

<Paste screenshot of query and output>

iv. Create a procedure to display Employee details with Department details.

<Paste screenshot of query and output>

v. create a function `employee_summary` that:

- Accepts `EmpID` as input.
- Returns the following details:
  - Employee Name
  - Department Name
  - Total number of dependents
  - Total annual salary (Salary × 12)
- The function must return a composite result (multiple columns).

<Paste screenshot of query and output>

b. Cursors

i. Create a cursor to display Employees of a Department with ID 101. Expected output is of the format:

'NOTICE: EmpID: 101, Name: Asha, Salary: 50000
NOTICE: EmpID: 101, Name: Anna, Salary: 20000'

<Paste screenshot of query and output>

ii. Create a cursor to list Employees with their employee ID and Department Name. Expected output is of the format :

'NOTICE: EmpID: 104, Name: Arjun, Department: Marketing
NOTICE: EmpID: 105, Name: Priya, Department: HR'

<Paste screenshot of query and output>

iii. Create a cursor to Count Dependents of Each Employee. Expected output is of the format:

'NOTICE: Employee ID: 104, Dependents: 1
NOTICE: Employee ID: 102, Dependents: 2'

<Paste screenshot of query and output>

iv. Create a cursor to display empid,salary and ename of two highest paid employees

<Paste screenshot of query and output>

RESULT

Implemented queries to create Packages and cursors in SQL and outputs were verified.

Date : **Page No:**

Experiment No.11

CRUD OPERATIONS ON A CASSANDRA TABLE

AIM

Perform basic CRUD (Create, Read, Update, Delete) operations on a Cassandra table.

QUERIES AND OUTPUTS

a. Create a keyspace named company.
<Paste screenshot of query and output>
b. Show all available keyspaces
<Paste screenshot of query and output>
c. Choose and use the keyspace created. Ie, company
<Paste screenshot of query and output>
d. Create the following three tables
department (did, dname, fund)
employee (eid, ename, dob, deptid)
dependent (empid, dependentname, age)
<Paste screenshot of query and output>
e. Insert atleast three rows in each table.
<Paste screenshot of query and output>
f. Read all the contents of each table.
<Paste screenshot of query and output>
g. Update the name of the employee with empid 101 as 'Anjali'
<Paste screenshot of query and output>
h. Delete the record of dependent of the employee whose empid is 102.
<Paste screenshot of query and output>
i. Delete a specific Dependent named 'Kiran' of Employee 101.
<Paste screenshot of query and output>

RESULT

Performed basic CRUD operations on a Cassandra table and outputs were verified.

Date : **Page No:**

Experiment No.12

CQL QUERIES ON CASSANDRA TABLES FOR DATA RETREIVAL

AIM

Write and execute CQL queries to retrieve specific data from Cassandra tables

QUERIES AND OUTPUTS

a. Display dependentname and age of all dependents in the company.
<Paste screenshot of query and output>
b. Display the names of all dependents of employee with ID 101.
<Paste screenshot of query and output>
c. List the Ids of employees who have dependents with age greater than 6.
<Paste screenshot of query and output>
d. Display the dependent table. But, limit the number of results returned to one record.
<Paste screenshot of query and output>
e. Display the details of all the dependents of an employee whose empid is 101 in the ascending order of dependentname.
<Paste screenshot of query and output>
f. Insert a new column 'gender' in dependent table. Update the values in each record also.
<Paste screenshot of query and output>
g. Drop the column 'gender' from dependent table.
<Paste screenshot of query and output>
h. Find Number of Dependents for an Employee whose id is 101
<Paste screenshot of query and output>
i. Display id and name of employees who were born after 1st January 1995.

<Paste screenshot of query and output>

j. Find the maximum age of the dependent of employee whose id is 101.

<Paste screenshot of query and output>

RESULT

CQL queries to retrieve specific data from Cassandra tables were executed and outputs were verified.

Date : **Page No:**

Experiment No.13

APPLICATION DEVELOPMENT USING MONGODB WITH PYTHON

AIM

Design a database application using any front-end tool for any problem selected. The application constructed should have five or more tables. Create a simple application using Mongodb with python.

<Give a summary of the application in a paragraph.Paste the design that includes more that 5 tables, code, necessary screenshots showing the working of the application>

RESULT

Designed and developed a database application using Mongodb with python and outputs were verified.